

District IQ

Review Document

Functionality · Algorithms · Tech Stack

How each is done, why it is used, and why it is better

Chennai Intelligent District Governance Platform · FarmwiseAI Campus Product Challenge, Task 6 · 1 October 2026

How to read this document

The document has three parts, matching the review questions:

Part	Answers the question	Format
Part 1 — Functionality	What does the system do, and how is each feature done?	Table: feature, what it does, how it is done (method), why
Part 2 — Algorithms	Which algorithms run, how they work step by step, why we chose them, why they beat the alternatives	One card per algorithm (A1–A20) with the real parameters from our code and measured results
Part 3 — Tech stack	Which technology, how we used it, why, why better	Tables grouped by layer, plus a one-page summary of design decisions

One-line summary of the method: independent collectors fetch real public data and generate realistic department records → a Python pipeline cleans, classifies, geo-locates, de-duplicates and links them into real-world incidents with explainable priority → statistics find spikes and hotspots → a Next.js web app shows role-based consoles → an AI assistant answers questions, and a verifier guarantees every number it says is in the data. The whole thing also runs on AWS.

Core principle used everywhere: numbers come from SQL and statistics, never from a language model. AI is only used to understand questions and write sentences, and its output is checked. This is why the system can be trusted by a Collector.

Part 1 — Functionality

Grouped by who uses it. The "How it is done" column names the method; the algorithm cards in Part 2 explain each in depth.

Feature	What it does	How it is done (method)	Why it matters
Automatic data collection	Fetches news, IMD weather and warnings, CPCB air quality, CFM-DSS river and lake levels, AGMARKNET prices; refreshes synthetic police, PWD, hospital and grievance data	One Python/Node program per source; HTTP + JSON APIs, RSS, Playwright for JS pages; 3 retries; scheduled daily 6 AM (PC) or hourly/3-hourly (GitHub Actions → API Gateway → Lambda)	Data is always fresh without manual work
News understanding	Keeps only Chennai news, tags the department, finds which articles are real incidents, groups the same story	Gazetteer filter (A1), keyword department tagging (A2), incident classifier (A4), story clustering (A5)	News shows problems no department has logged
Unified incident view	One row per real-world incident even when reported by grievance, police, PWD and news	Category classifier (A3), geo-location (A6), dedup (A8), cross-source linking (A9)	Removes duplicate work; shows true scale
Priority with reasons	Ranks incidents and says why ("past the 24 h target; 3 sources; rain-linked")	Rule-based severity + additive priority score (A10)	Collector sees what needs attention and the reason
Spikes and hotspots	Flags unusual daily counts by category and zone; finds clusters and hot wards	Poisson baseline (A11), DBSCAN (A12), Getis-Ord Gi* (A13), EWMA/z-score (A14)	Early warning before problems grow
Briefings and gaps	Daily/weekly/monthly/quarterly briefings; issues seen only in news	Rule agents with a numeric verifier (A15), gap finder (A16)	Ready-to-read summary, every number true
Citizen portal	Sign-up with OTP, file complaint with map pin and photo, track status	Next.js forms + zod validation, Leaflet map, ward from polygons, AI/keyword category for "Others"	Single entry point for citizens
Collector console	4 fixed-screen pages: Overview (satellite map, snapshot, severity, My Tasks, briefing), Briefing & stories, Trends, Environment & markets	React components reading SQL views; filters for period/department/zone/taluk; incident pop-up with plain reasons	One screen with the whole district
Decisions & audit	Verify or send back officers' work; every action logged	Writes to ops store (collector_decisions, audit_log), never overwritten by the pipeline	Accountability and traceability
Department consoles (25)	Each officer sees only their department: approve → act → complete with photo → Collector verifies	Department taken from the database each request; workflow state machine; per-department insight cards	Work tracked to verified completion
Exports	PDF briefing report and CSV action list	jsPDF + autotable in the browser; CSV from the same queries	Shareable in meetings
Ask District IQ	Chat in English, Tamil, Tanglish; answers with charts, maps, tables; follow-up edits; insights for today	Guard → router → tools / safe SQL compiler → facts → composer → number verifier → chart checker (A17–A19)	Ad-hoc questions answered in seconds, with no made-up numbers
Data quality	Source health, freshness chip, quarantine, drift, location validation, evaluation report	Data Steward agent + validation scripts	Problems are visible, not hidden
Cloud (AWS)	Collectors as Lambdas, data on S3/DynamoDB, AI on Bedrock	Diff-only writes, content-addressed snapshots (A20)	Runs in the challenge's AWS account at low cost

Part 2 — Algorithms

Each card gives the algorithm's purpose, the exact method and parameters used in our code, why we chose it, why it is better than the common alternative, and the measured result where we have one.

A1. Chennai relevance filter (gazetteer matching)

Used in	chennai_news_pipeline/src/relevance_filter.py
Purpose	Keep only news about Chennai district from thousands of RSS items.
How it is done	A gazetteer (list of Chennai places, localities and landmarks in English and Tamil script, with spelling variants) is matched against the headline and text. Exclusion rules drop other districts and national stories. Language is detected with langdetect first so Tamil and English rules are applied correctly.
Why we used it	News feeds mix national and other-district news; the dashboard must show Chennai only.
Why it is better	vs a trained location model: no labelled training data was available, a gazetteer is fully explainable, handles Tamil script directly, and can be fixed by adding a place name (then --reprocess, no re-fetching).

A2. Department tagging of news (keyword model)

Used in	chennai_news_pipeline/src/classification.py
Purpose	Tag each article with one of 15 departments and a complaint flag so it can be routed.
How it is done	Weighted keyword lists per department (English + Tamil) are matched; the highest score wins; a complaint flag is set when civic-failure words appear. Also used in the AWS news Lambda.
Why we used it	Needed on day one, before any labels existed; must run inside a small Lambda.
Why it is better	vs an LLM per article: free, instant for 20,000+ articles, deterministic and testable (88 tests); no API cost or quota.

A3. Category classifier — character n-gram TF-IDF + Logistic Regression

Used in	district_intel/dintel/classify.py
Purpose	Put free text ("Other" complaints, news) into the standard category list.
How it is done	Text → TF-IDF on character n-grams (analyzer char_wb, n = 2 to 5, sublinear tf, up to 200,000 features) → Logistic Regression (C = 6, class_weight = balanced). Trained on labelled grievance, police and PWD text. Predictions below 0.45 confidence stay "Other". Structured sources use crosswalk tables instead.
Why we used it	Complaint text is short, noisy and mixes Tamil, Tanglish and spelling errors.
Why it is better	Character n-grams match parts of words, so "drainage", "drainge" and "draineage" look alike — word models and keyword lists miss these. Much smaller and faster than BERT, runs on a laptop CPU, and gives probabilities for the confidence cut. Balanced class weights stop big categories dominating.
Result	About 0.99 accuracy on template text (we expect lower on real citizen text and say so).

A4. News incident filter (weak supervision + hand labels)

Used in	district_intel/dintel/loaders/news.py
Purpose	Decide whether a news article reports a real incident a department must act on (flooding, accident, collapse) or not (power-cut schedule, politics, court case).
How it is done	1) Weak labels from rules give a large, noisy training set. 2) 300 hand labels from stratified sampling (150 accepted, 75 just below threshold, 75 far below). 3) A classifier is trained on both. 4) Scored with 5-fold cross-validation , population-weighted by stratum, so it is never scored on its own training labels.
Why we used it	Most news is not an incident; showing all of it would bury the Collector.
Why it is better	vs rules only: F1 rose from 0.41 to 0.63. vs labelling thousands by hand: weak supervision gets most of the value from 300 labels. Stratified sampling focuses labels where the model is unsure.
Result	F1 0.63 (precision 0.69, recall 0.57). More labels is the known path to improve.

A5. Story clustering (two-stage: n-grams, then multilingual embeddings)	
Used in	district_intel/dintel/loaders/news.py, dintel/embed.py
Purpose	Group articles about the same event ("developing stories"), and count distinct outlets.
How it is done	1) One document per URL; Google News copies merged into the publisher's copy. 2) Stage 1: character n-gram similarity catches near-identical headlines cheaply. 3) Stage 2: multilingual-e5-small sentence embeddings; two articles join if cosine similarity ≥ 0.95 within 24 hours . 4) Groups formed with union-find.
Why we used it	The same event is rewritten by many outlets; counting each as a new incident would inflate numbers.
Why it is better	Embeddings catch rephrased headlines that n-grams miss; the cheap first stage cuts the embedding work. e5-small is free, runs offline on CPU and understands Tamil — vs paid cloud embeddings. The strict 0.95 cut prefers missing a merge over a wrong merge.
Result	129 same-language merges, all checked by hand and correct. English–Tamil merging is switched off because headline similarity could not separate same-event pairs from different accidents on the same day.

A6. Geo-location: point-in-polygon, snapping, taluk majority vote	
Used in	district_intel/dintel/geo.py, geolocate.py
Purpose	Put every record into the correct ward, zone and taluk so maps and filters are right.
How it is done	1) Records with coordinates: point-in-polygon test against the 200 GCC ward polygons (matplotlib.path). 2) Points within 500 m outside a ward are snapped in and flagged (confidence 0.8). 3) Records without coordinates: resolved from street (± 300 m), locality (± 800 m) or area (± 1500 m) using the gazetteer, with that precision stored. 4) Ward → taluk by majority vote of police and PWD points that carry a taluk code. Low confidence (< 0.7) goes to review.
Why we used it	Sources use different location formats (lat/lon, ward number, street text, taluk).
Why it is better	Real ward geography instead of invented regions. matplotlib.path needs no GEOS/GDAL install (a common Windows problem with Shapely/GeoPandas). Snapping fixes GPS points just outside a boundary instead of losing them. Storing precision lets later steps (linking) widen their search for vague locations.

A7. Scenario overlay (one shared world for synthetic data)	
Used in	district_intel/dintel/world.py, dintel/sync.py
Purpose	Make separately generated synthetic sources agree with each other and with real weather.
How it is done	1) A world calendar collects rain days from every source (grievances, police events, PWD flood spikes, IMD warnings). 2) The same rain days are passed to every generator through its own options. 3) On a world rain day, a source with no flood response gets the reports it would have produced, sized from its own real rain days; existing records are adopted, not duplicated. 4) Every planted row is flagged <code>is_synthetic = 1</code> , <code>is_overlay = 1</code> ; a truth file records which world event it belongs to.
Why we used it	Before this, each generator invented its own rain days (flood-day correlation ≤ 0.12), so cross-source linking could not be tested.
Why it is better	Generators stay untouched (project rule); real sources are never changed; the truth file gives us ground truth to measure dedup and linking accuracy — which most prototypes cannot do.
Result	Flood agreement per 3-day episode: 0.67 / 0.64 / 0.91.

A8. Duplicate removal — blocking + union-find	
Used in	district_intel/dintel/dedup.py
Purpose	Merge repeated reports from the same source (e.g. 5 citizens reporting one pothole).
How it is done	1) Blocking : only compare records with the same category, nearby and close in time. 2) Source-specific rules decide a duplicate (portal's rule for grievances, officers' own "Duplicate of" decisions, police and PWD rules). 3) Union-find (disjoint-set) joins all duplicate pairs into groups.
Why we used it	Duplicates overstate the problem and create double work.
Why it is better	Blocking avoids comparing every pair ($n^2 \rightarrow$ near-linear). Union-find is transitive ($A=B$ and $B=C \Rightarrow A=C$) in almost $O(1)$ per merge, and keeps every member traceable — unlike simply deleting duplicates.

A9. Cross-source linking — KD-tree blocking + Logistic Regression + learned threshold	
Used in	district_intel/dintel/linking.py
Purpose	Recognise that a grievance, a police report, a PWD incident and a news story are the same real-world incident.
How it is done	1) Blocking : compatible category groups only (e.g. FLOOD and DRAINAGE both = WATER), within each category's own time window (+48 h for news) and radius (+ both points' location precision). A KD-tree (scipy cKDTree) finds spatial neighbours. 2) Pair features : normalised distance, time gap, TF-IDF text similarity, same category, same ward, coarse location, has-news. 3) Logistic Regression (balanced) trained on half of the world events; the other half is held out for testing. 4) Merge threshold chosen to maximise pair F1 on the training half. 5) Decision bands : ≥ 0.85 link; $0.40\text{--}0.85 \rightarrow$ review queue (Linker agent, then a person); < 0.40 no link. 6) Union-find builds incidents. Each pair stores its features for explanation.
Why we used it	Record IDs never match across departments (CMP-..., GRV-..., 2026-NNN), so linking must use place, time and text.
Why it is better	KD-tree makes cost grow with neighbours, not n^2 . Logistic regression gives a calibrated probability and readable weights — a reviewer can see why two records were linked, unlike a neural network. The gray band sends uncertain cases to people instead of guessing. Held-out evaluation proves the number.
Result	Precision 0.99, F1 0.88 on held-out events.

A10. Severity and explainable priority score	
Used in	district_intel/dintel/severity.py, incidents.py
Purpose	Rank incidents for the Collector and explain the rank in plain words.
How it is done	Severity (static facts only, 0–100): category risk (e.g. flood / high-risk issue +35, administrative +5), plus facts from the record, e.g. people injured (+8 each, up to +24) or dead. Priority = severity + additive bonuses, each adding a reason. Deadlines come from each category's target hours (response-based for police, resolution-based for others).
Formula / rule	$\text{priority} = \text{severity} + 5 \cdot \log_2(\text{sources}) + 3 \cdot \log_2(1 + \text{citizen complaints}) + 10 \text{ if past deadline (20 if over } 2\times) + 8 \text{ if Severe/High and not verified} + 8 \text{ if } \geq 3 \text{ new reports in 24 h} + 5 \text{ each for: vulnerable people affected, } \geq 2 \text{ news outlets, only in news, rain-linked, } \geq 3 \text{ similar incidents in 90 days.}$
Why we used it	Collectors need a short list of what to act on, and must trust it.
Why it is better	vs a black-box ML ranking: every point is explainable and auditable (shown as reasons in the pop-up); no training data needed; weights are tunable in YAML. Priority is recomputed at as-of time so it never goes stale. $\log_2$ scaling stops one viral issue with 100 complaints drowning everything else.

A11. Spike detection — Poisson baseline with weekday factor	
Used in	district_intel/dintel/analytics.py (anomalies)
Purpose	Detect an unusual rise in incidents of a category in a zone on a day.
How it is done	1) Daily counts per (category $\times$ zone). 2) Baseline λ = rolling 28-day mean of previous days (needs ≥ 14 days), multiplied by a district-wide weekday factor (some weekdays are always busier than others). 3) $p\text{-value} = P(X \geq \text{observed} \mid \text{Poisson}(\lambda))$. 4) Spike if $p < 0.01$ and observed ≥ 3 .
Why we used it	Counts of events per day follow a Poisson distribution; we need an early warning, not just a chart.
Why it is better	vs a fixed threshold ("more than 10"): adapts to each zone and category. vs mean + 2·SD: correct for small counts, where the normal approximation fails. The weekday factor stops normal busy-weekday peaks being flagged. The minimum count of 3 avoids alarms from $0 \rightarrow 1$. Output includes observed, expected, ratio and p-value — fully explainable.

A12. Hotspots — DBSCAN with haversine distance	
Used in	district_intel/dintel/analytics.py (hotspots)
Purpose	Find places where incidents of one category keep happening (e.g. a junction that floods every rain).
How it is done	Per category, last 90 days of incident points → DBSCAN with haversine metric (earth distance), eps = 250 m , min_samples = 5 , ball-tree index. Each cluster becomes a hotspot with its centre and count.
Why we used it	Repeated problems need a permanent fix, not repeated repairs.
Why it is better	vs K-means: no need to choose the number of clusters, finds any shape, and ignores scattered noise points. Haversine gives true metres on the earth's surface. 250 m is about a street block, matching a field officer's site.

A13. Hot wards — Getis-Ord Gi* spatial statistic	
Used in	district_intel/dintel/analytics.py (gi_star, ward_stats)
Purpose	Find wards that are statistically hot together with their neighbours, not just high on their own.
How it is done	Incident counts per ward over 30 days; neighbours from ward polygon contiguity (self included). Gi* z-score compares the local sum with the district mean. A ward is hot when z > 1.96 (95% confidence).
Why we used it	A raw count map is misleading: big wards naturally have more incidents.
Why it is better	Gives statistical significance instead of eyeballing a colour map; standard method in spatial crime and health analysis; complements DBSCAN (point clusters) with area-level evidence.

A14. EWMA and z-score for sensor series	
Used in	district_intel/dintel/analytics.py (observation_signals)
Purpose	Show whether lake levels, AQI, rainfall, hospital beds are unusually high or low now.
How it is done	For each metric series: EWMA with span 7 (smoothed trend) and the z-score of the latest value against the last 28 days' mean and standard deviation. Quality flags mark gaps and suspicious readings.
Why we used it	Sensor readings are noisy; a single reading is not a trend.
Why it is better	EWMA reacts faster than a simple moving average while still smoothing noise; z-score gives one comparable scale for different units (metres, AQI, %).

A15. Agents with a numeric verifier (Briefing, Steward, Planner, Watchdog)	
Used in	district_intel/dintel/agents/
Purpose	Write daily/weekly/monthly/quarterly briefings, draft action plans from playbooks, check data health.
How it is done	Agents are rule-based programs that read the computed tables and fill templates. If an LLM is enabled, it may only rewrite the text, and the result is accepted only if the numeric verifier finds every number in the source data. Calls are capped per run.
Why we used it	Collectors want readable text, but cannot accept wrong figures.
Why it is better	vs letting an LLM write the briefing: no hallucinated numbers; works with no API key; cheap. The system degrades gracefully: every LLM step has a deterministic fallback.

A16. Gap finder ("Only in news")	
Used in	district_intel/dintel/agents/workers.py; Collector page 2
Purpose	Show problems reported in the news that no department has recorded.
How it is done	A news incident that the linker could not join to any department record (media_only) becomes a gap; it gets +5 priority and is auto-routed to the right department's officer by its tag.
Why we used it	This is the blind spot a Collector most wants to know about.
Why it is better	Uses the linking result directly, so no extra model is needed; turns news from "reading" into an action list.

A17. Ask District IQ — guarded tool pipeline with SQL compiler	
Used in	chennai-grievance-portal-main/src/lib/assistant/ (pipeline, guard, compile, tools)
Purpose	Answer free questions safely, with real numbers.
How it is done	1) Guard (code, before any model): refuses personal-data requests, data changes, prompt-injection attempts. 2) Router : a fast model (Bedrock Nova Micro / Ministral) picks a console tool; without a key, rules pick it ("Without AI"). 3) For new groupings, the model fills a QueryPlan (it never writes SQL); the compiler turns it into one parameterised SELECT using only catalogued tables, one-hop declared joins, a 5 s time limit, row caps, read-only connection; errors go back to the planner for at most 2 repairs. 4) Facts → composer writes the answer. 5) Officers: lockDept forces their department on every tool and query on the server.
Why we used it	Collectors ask questions the fixed dashboard does not answer.
Why it is better	vs text-to-SQL (model writes SQL): no SQL injection, no table outside the catalog, no runaway query. vs a plain chatbot: answers come from data, not model memory. Department lock is enforced server-side, so a prompt cannot bypass it.

A18. Number verifier and chart checker	
Used in	src/lib/assistant/verify.ts, chartspec.ts
Purpose	Guarantee that no number in an AI answer is invented.
How it is done	Every number in the headline, text and chart titles is extracted and must match a returned fact at the precision written ("97.7%" and "98%" both match 97.65). Allowed context: dates, times, IDs, zone/ward numbers, window lengths (7, 30, 90 days), numbers in the question. Anything else fails the answer. The chart spec is validated before display.
Why we used it	LLMs can produce confident but wrong numbers — unacceptable for government decisions.
Why it is better	Most chatbots trust the model; ours proves each number. This is the key trust feature of the assistant.
Result	Covered by the 186 Vitest tests and a golden-question evaluation set.

A19. Typo and Tanglish handling — Damerau-Levenshtein spell correction	
Used in	src/lib/assistant/fuzzy.ts, spell.ts, lang.ts
Purpose	Understand "incidnets in adyr" or "thakkali vilai" (tomato price).
How it is done	Edit distance with swaps counted as one edit (Damerau-Levenshtein , early stop past 3). Words are corrected only toward the district's own vocabulary (places, categories, commodities, console terms), only when one candidate is clearly closest, and never when the word is a real English word (274,000-word list) or romanised Tamil. Tamil, Tanglish and Hindi commodity aliases are mapped. The answer shows "Understood as ...".
Why we used it	Officers type quickly, in mixed languages.
Why it is better	vs generic spell-checkers: they would "correct" Chennai place names into English words. Refusing to guess between equal candidates avoids wrong corrections; showing the correction keeps it transparent.

A20. Diff-only writes, atomic swap and content-addressed snapshots	
Used in	aws/lambda/collect, aws/export_intel.py, dintel/mysql_export.py
Purpose	Update data stores quickly, cheaply and without readers ever seeing half-loaded data.
How it is done	MySQL : load into *_new tables, then one atomic RENAME swaps all at once; row and column counts are compared with SQLite. DynamoDB : each Lambda writes only rows that changed since last run. S3 : each table is gzip JSON named by its SHA hash; unchanged tables are skipped; a manifest (intel/current.json) is switched last.
Why we used it	Data refreshes every hour while people are using the dashboard.
Why it is better	No downtime and no partial reads; far fewer writes (lower AWS cost); hashes make uploads idempotent and safe to retry after expired credentials.
Result	31 tables, 19.4 MB compressed per snapshot; unchanged tables cost nothing.

Part 3 — Tech stack: how we used it, why, and why it is better

Grouped by layer. "How we used it" describes our actual method in the project.

Data collection layer

Technology	How we used it (method)	Why we used it	Why better than alternatives
Python 3.11	All collectors, news pipeline, district_intel and AWS scripts	Best data and ML ecosystem	vs Java/Node: far richer data/geo/ML libraries; less code
requests + feedparser	HTTP calls to IMD/CFM/AGMARKNET JSON APIs; parsing Google News and publisher RSS	Simple, reliable, key-free	vs paid news APIs: free, no quota, includes Tamil outlets
trafilatura	Extracts article body from news pages	Best open-source article extractor	vs BeautifulSoup by hand: removes menus/ads automatically for any site
Playwright	Reads CPCB's JavaScript-rendered public AQI map	Page has no open API	vs Selenium: auto-waits, faster, fewer flaky runs
langdetect, tzdata, python-dateutil	Detect Tamil/English; convert all times to Asia/Kolkata	Correct language rules and IST times	vs manual parsing: handles many date formats and Windows time zones
truststore	Uses the Windows certificate store for HTTPS	Antivirus inspects TLS on this PC	vs verify=False: stays secure
Node.js + TypeScript	Police dataset generator	Same language as the portal	vs plain JS: types catch errors at build time

Intelligence layer

Technology	How we used it (method)	Why we used it	Why better than alternatives
pandas / NumPy	Load 8 sources into one 70-column schema, joins, group-bys, rolling windows	Fast columnar data work	vs loops/Excel: much faster, repeatable, testable
scikit-learn	TF-IDF char n-grams, Logistic Regression (classifier and linker), DBSCAN, cross-validation	Proven, explainable ML	vs deep learning: works on small data, CPU-only, explainable weights
SciPy	Poisson p-values, normal z cut-offs, cKDTree spatial search	Correct statistics and fast neighbour search	vs hand-written maths: tested, accurate, O(log n) lookups
sentence-transformers (multilingual-e5-small)	Embeddings for English and Tamil headlines to cluster stories	Multilingual, small, offline	vs OpenAI embeddings: free, private, no internet at build time
matplotlib.path	Point-in-polygon for 200 ward shapes	No native GIS install	vs Shapely/GeoPandas: no GEOS/GDAL setup problems on Windows
PyYAML	Categories, deadlines, link windows, playbooks, thresholds in config.yaml / reference/*.yaml	Tuning without code changes	vs hard-coded constants: reviewers can read and change rules
SQLite	Pipeline output store (32 tables, 6 views); in-memory store in the AWS website	Zero-config single file	vs a DB server for batch output: portable, nothing to install
pytest	24 district_intel + 88 news pipeline tests	Catch regressions	vs manual checks: runs in seconds after each change

Application layer

Technology	How we used it (method)	Why we used it	Why better than alternatives
Next.js 14 (App Router) + React 18	Citizen, Collector and officer pages and all REST APIs in one app; middleware for role-based routes	Full-stack in one project	vs React + separate Express: one deployment, shared types, server rendering
TypeScript	All portal code	Type safety on a large codebase	vs JavaScript: errors caught before running

Technology	How we used it (method)	Why we used it	Why better than alternatives
Tailwind CSS + scoped CSS	Portal styling; fixed 1366×680 console canvas that scales	Fast consistent UI	vs Bootstrap: lighter, fully custom design
MySQL 8 + mysql2	Store for the non-AWS version; parameterised SQL only, no ORM	Concurrent reads/writes	vs ORM: transparent, fast SQL; vs string SQL: no injection
Leaflet + OpenStreetMap / Esri imagery	Complaint pin map; Collector satellite map with ward and zone outlines	Free maps, no key	vs Google Maps: no billing account or API key
ECharts	Charts in assistant answers	Many chart types, fast	vs Chart.js: richer types, handles more data
jsPDF + jspdf-autotable	Briefing PDF built in the browser	No server load	vs server PDF: instant, works offline
bcryptjs, jose (JWT), zod	Password hashing, httpOnly JWT cookies, input validation on client and server	Standard security	vs tokens in localStorage: protected from XSS
nodemailer	OTP and password-reset emails	Simple SMTP	vs paid email services: no lock-in
tesseract.js	Collector uploads a scanned newspaper page or notice; OCR reads English + Tamil in the browser and adds the items as a source	Runs in the browser, free	vs cloud OCR (Texttract): not allowed and costs money
Vitest	186 tests for compiler, verifier, chart checker, guards, typos	Fast TS testing	vs Jest: faster, native ESM/TypeScript

AI layer

Technology	How we used it (method)	Why we used it	Why better than alternatives
Vercel AI SDK (ai, @ai-sdk/*)	Streaming answers; switch Bedrock / Groq / OpenAI by config	One API for many models	vs separate SDKs: change provider without code changes
Amazon Bedrock (Nova Micro, Nova Lite, Minstral 3B/8B)	Router (fast, cheap model) and composer	Allowed models; data stays in AWS	vs OpenAI: in-region, billed to the team account, no data leaves AWS
@huggingface/transformers	Multilingual embeddings inside Node for matching questions	Local and free	vs hosted embeddings: no cost, private
word-list	274,000 English words so the spell-checker does not "correct" real words	Avoid wrong corrections	vs no list: fewer false corrections

Cloud and operations layer

Technology	How we used it (method)	Why we used it	Why better than alternatives
AWS Lambda	5 collector functions from one package (SOURCE env), news, intel and store functions	Serverless, pay per run	vs EC2: no idle cost, no server to patch (EC2 not allowed)
Amazon API Gateway (HTTP API)	POST /refresh/<src>, /ingest, /intel, /store with x-refresh-key; 1 req/s, burst 5	Secure entry point	vs public Lambda URLs: key check and throttling
Amazon S3	Raw files, content-addressed snapshots, uploads, exports	Cheap, durable	vs storing blobs in a DB: much cheaper for big files
Amazon DynamoDB	Collector rows (diff-only), users, complaints, ops tables	Serverless key-value store	vs RDS: not allowed and needs a running server
CloudWatch Logs	Lambda logs and errors	Built-in monitoring	vs custom logging: zero setup
GitHub Actions	Cron: hourly IMD/CFM/PWD/hospital, news every 3 h, via API Gateway	Free scheduler	vs EventBridge: denied for our AWS role
Windows Task Scheduler	Daily 6 AM district_intel build on the PC; pushes CPCB and police	No server cost	vs always-on VM: free; ML build too big for Lambda

Technology	How we used it (method)	Why we used it	Why better than alternatives
Git + GitHub	Two repos: MySQL version and AWS version	Version control, review	Keeps both versions clean and separate

Key design decisions (summary for the reviewer)

Decision	Alternative we rejected	Why our choice is better
Numbers from SQL and statistics; AI only for language	Let the LLM compute and write numbers	No hallucinated figures; a verifier proves every number
Explainable models (logistic regression, rule scores)	Deep neural networks	Small data, CPU-only, and reviewers can see why
Cross-source linking with a human review band	Fully automatic merging	Uncertain cases go to people; precision 0.99
Collectors never modified; fixes in a separate layer	Edit each collector for the dashboard	Sources stay testable; one place to fix data
Pipeline store vs dashboard ops store	One shared database	Rebuilds never erase people's decisions
Real ward geography (200 GCC wards)	Invented regions	Maps and filters match how Chennai is governed
Measured accuracy with held-out ground truth	No evaluation	We can state numbers (F1 0.88, 0.63) and limits honestly
Serverless AWS with external timer	Servers / EventBridge	Fits allowed services and budget